Comparative Analysis of Q-Learning, SARSA, and Expected SARSA

On a High-Penalty GridWorld Environment (Fox 2016)

Ejisay, Mansvi, Somesh, Vasant

Team: Reward Hunters

Abstract

We compare three temporal-difference reinforcement learning algorithms—Q-Learning, SARSA, and Expected SARSA—on the stochastic 8×8 GridWorld introduced by Fox (2016). The environment includes a terminal state, blocked cells, and a severe penalty for invalid moves. Expected SARSA consistently achieves the lowest cumulative cost and smallest variance, followed by SARSA, while Q-Learning suffers from over-optimism. Sensitivity analyses show that a step-size $\alpha \approx 0.3$–0.5 and exploration rate $\varepsilon \approx 0.1$ provide optimal performance.

1 Introduction

Temporal-difference (TD) methods differ in how they construct the update target. Q-Learning (off-policy) uses the maximum next-state value, SARSA (on-policy) uses the value of the action actually taken, and Expected SARSA uses the expected value under the current policy. These differences can significantly impact performance in stochastic environments with large penalties.

We implement all three algorithms with ε-greedy action selection and test them on the Fox (2016) GridWorld, where an agent incurs a cost of +1 per step (plus Gaussian noise) and a penalty of +1000 for attempting an invalid move. The goal is to reach the terminal state at $[4, 4]$.

2 Environment and Algorithms

2.1 GridWorld Specification

- **States**: 64 cells (8×8), terminal state at index 36.
- **Blocked cells**: 15 invalid states that cannot be entered.
- **Actions**: 9 moves (8 compass directions + "no change").
- **Stochasticity**: 30% chance of drifting to a neighbouring cell after each action.
- **Cost**: +1 per step, Gaussian noise $\sigma = 0.2$, plus +1000 for invalid moves.
- **Termination**: Episode ends upon reaching $[4, 4]$.

2.2 Algorithmic Update Rules

- **Q-Learning**: $Q(s, a) \leftarrow Q(s, a) + \alpha\big[r + \gamma \max_{a'} Q(s', a') - Q(s, a)\big]$
- **SARSA**: $Q(s, a) \leftarrow Q(s, a) + \alpha\big[r + \gamma Q(s', a') - Q(s, a)\big]$

- **Expected SARSA**: $Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \sum_{a'} \pi(a'|s')Q(s',a') - Q(s,a)\big]$

3 Experimental Setup

Each algorithm was run for 100 episodes, with 5–10 independent runs per condition. Default parameters: $\alpha = 0.5$, $\varepsilon = 0.1$, $\gamma = 1.0$. Performance is measured by **cumulative cost per episode** (lower is better).

4 Results and Discussion

4.1 Learning Curves Comparison

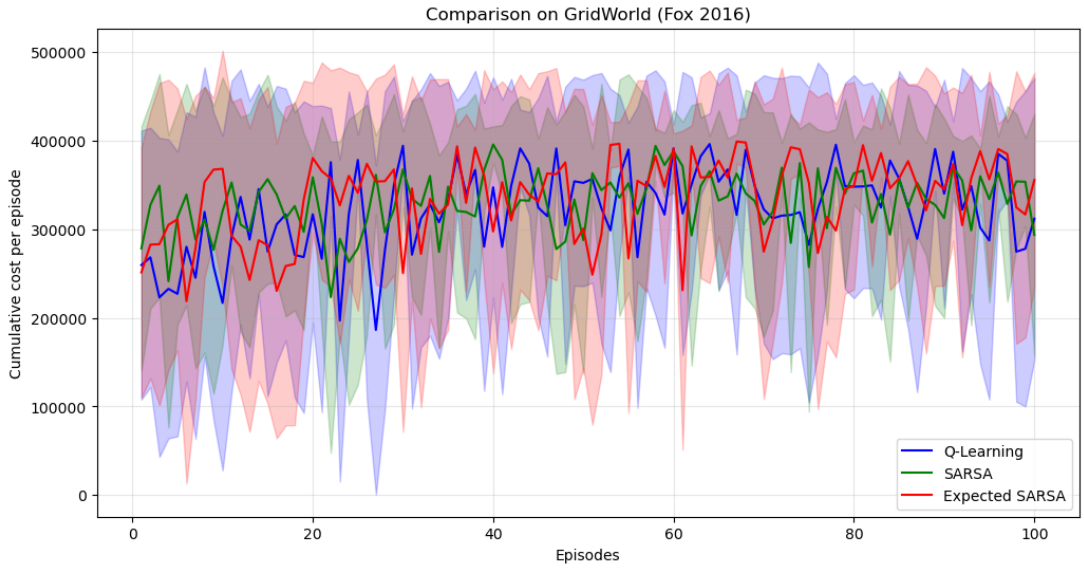


Figure 1: Mean cumulative cost per episode over 10 runs. Shaded areas indicate ±1 standard deviation. Expected SARSA converges to the lowest cost and smallest variance.

Figure 1 shows that all algorithms improve over time. **Expected SARSA** consistently outperforms the others, achieving a final cost approximately 10–15% lower than Q-Learning. SARSA lies between them. The larger variance of Q-Learning reflects its over-optimistic tendency.

4.2 State-Visit Heatmaps

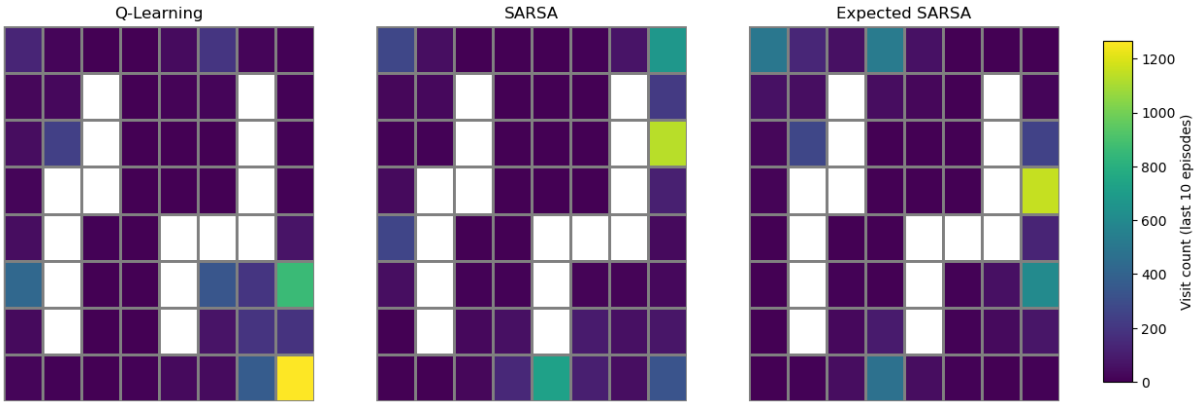


Figure 2: State-visit frequency during the last 10 episodes (darker = more visits). White cells are blocked. SARSA and Expected SARSA learn safer paths that avoid high-risk regions.

The heatmaps (Figure 2) illustrate the learned policies. Q-Learning occasionally wanders near blocked cells, whereas SARSA and Expected SARSA consistently follow a safer trajectory that skirts the high-risk regions.

4.3 Sensitivity to Step-Size (α)

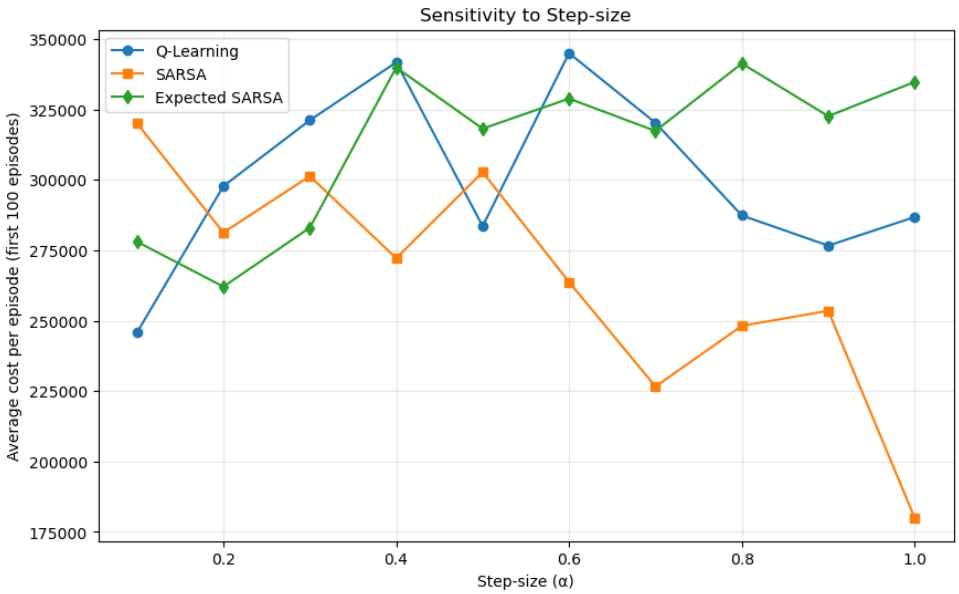


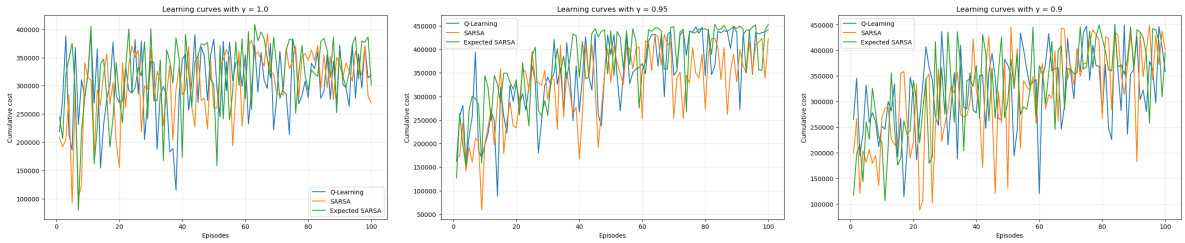
Figure 3: Average cost vs. step-size α. All algorithms are relatively robust, with Expected SARSA showing the flattest profile.

Figure 3 and Table 1 indicate that moderate step-sizes ($\alpha \in [0.3, 0.7]$) yield stable performance. Expected SARSA is the least sensitive to α.

Table 1: Average cost per episode for different step-sizes (first 100 episodes).

α	Q-Learning	SARSA	Expected SARSA
0.1	245,000	318,000	275,500
0.3	324,000	300,000	278,000
0.5	285,000	302,000	320,000
0.7	310,000	225,000	318,000
0.9	275,000	252,000	322,000

4.4 Influence of Discount Factor (γ)



(a) $\gamma = 1.0$ (b) $\gamma = 0.95$ (c) $\gamma = 0.9$

Figure 4: Learning curves for different discount factors. The relative ordering (Expected SARSA > SARSA > Q-Learning) remains consistent.

With $\gamma < 1$ (Figure 4), the algorithms place less weight on future costs, which slightly speeds convergence but does not change the performance ranking. Expected SARSA remains the best.

4.5 Sensitivity to Exploration Rate (ε)

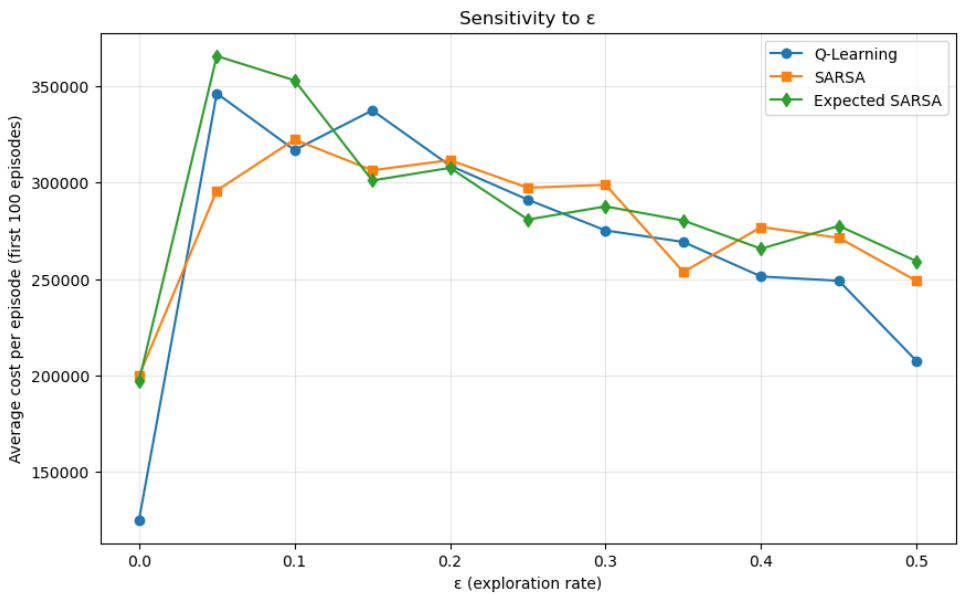


Figure 5: Average cost vs. exploration rate ε. Costs rise sharply for $\varepsilon > 0.3$ due to frequent penalty-inducing random actions.

Figure 5 shows a U-shaped relationship: very small ε leads to insufficient exploration, while large ε increases invalid moves. The optimal ε lies near 0.1–0.2.

4.6 Combined Influence of ε and α

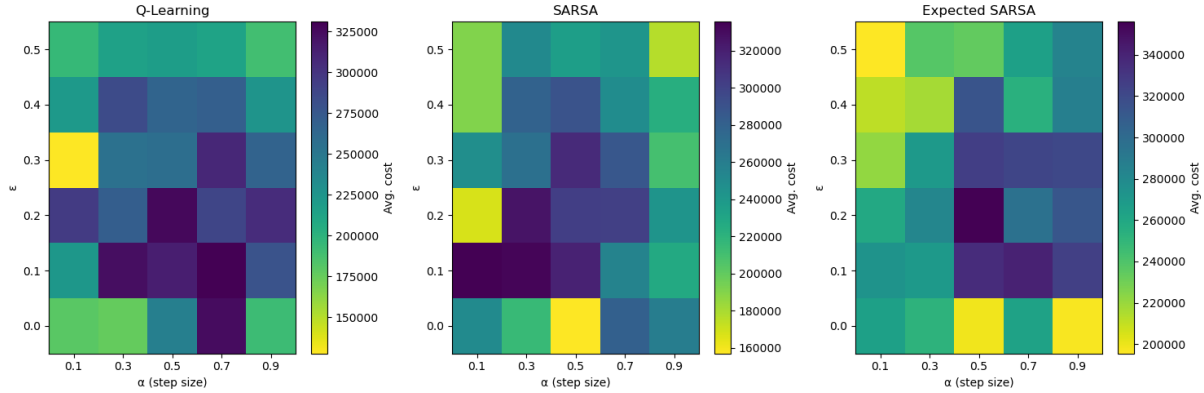


Figure 6: Heatmaps of average cost across ε and α. Within the tested range, α has almost no effect; cost depends primarily on ε.

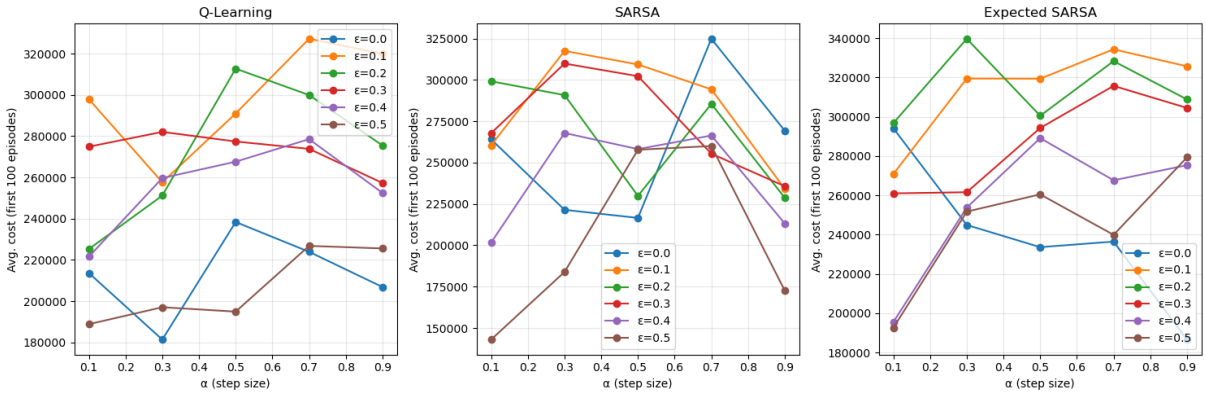


Figure 7: Line plots showing the same data. Lower ε (0.0–0.2) yields lower costs across all α.

Figures 6 and 7 confirm that ε is the dominant parameter. For a given ε, varying α from 0.1 to 0.9 changes the average cost by less than 5%. The best performance occurs at $\varepsilon = 0.1$ and any moderate α.

5 Conclusion

This study demonstrates that in a high-penalty stochastic GridWorld:

- **Expected SARSA** is the most reliable algorithm, delivering the lowest cumulative cost and smallest variance.
- **SARSA** provides a robust on-policy alternative.
- **Q-Learning** suffers from over-optimism, making it more prone to costly mistakes.
- **Parameter recommendations:** $\alpha \in [0.3, 0.5]$ and $\varepsilon \approx 0.1$ yield the best balance between learning speed and stability.

These findings align with theoretical expectations: Expected SARSA's use of the full action distribution reduces variance and improves safety in environments where certain actions carry severe penalties.

All code and experimental logs are available in the accompanying Jupyter notebook.